UNIVERSIDAD AUTÓNOMA DEL ESTADO DE HIDALGO

Instituto de Ciencias Básicas e Ingeniería (ICBI)

LICENCIATURA EN CIENCIAS COMPUTACIONALES

PRÁCTICA 5

ANÁLISIS LÉXICO DE ELEMENTOS DE UN LENGUAJE DE PROGRAMACIÓN

Asignatura:

Automatas y Compiladores

Alumna: Pilar Bucio Hidalgo

Docente: Eduardo Cornejo Velazquez

Semestre y grupo: 6ª3

1. Introducción

Algunas de las funciones del análisis léxico son

- Lee caracteres.
- Produce componentes léxicos (tokens).
- Filtra comentarios.
- Filtra separadores múltiples (espacios, tabuladores y saltos de línea).
- Lleva el contador de línea y columna del texto fuente.
- Genera errores en caso de que la entrada no corresponda a ninguna categoría léxica.

Para realizar dichas funciones, es importante tomar en cuenta los siguientes conceptos:

Categoría léxica. Tipo de símbolo elemental del lenguaje fuente, (identificadores, palabras clave, constantes numéricas, operadores, ...).

Componente léxico (token). Elemento perteneciente a una categoría léxica.

Lexema. Cadena de caracteres correspondiente al componente léxico.

Además, las categorías léxicas más comunes son:

Palabras clave. Palabras con un significado especial en el lenguaje

```
(if, then, else, for, while, do, switch, case, ...)
```

También se nombran como palabras reservadas.

Identificadores. Nombres de variables, de constantes, de funciones, de tipos...

```
(media, valor0, i1, _PI, ...)
```

Operadores. Símbolos que identifican operaciones aritméticas y lógicas.

```
( + , - , * , / , % , = , ++ , -- , += , -= , *= ,
/= , ==, !=, && , || , & , | , ...)
```

Constantes numéricas. Literales que especifican valores numéricos.

```
(325, 3.141592, 0xA3F2, 0.2e+3)
```

Constantes de carácter o cadenas. Literales con el valor de un carácter o de una cadena

```
('z', '\n', "ejemplo de cadena", ...)
```

Símbolos especiales. Separadores, delimitadores, terminadores, etc.

```
({ , } , [ , ] , ( , ) , ; , ,)
```

Blancos. Caracteres de separación de componentes léxicos

```
(espacios, tabuladores, saltos de línea)
```

El análisis léxico se limita a suprimirlos.

Comentarios. Información para el lector del programa.

```
/* comentario multilinea */, // comentario de una línea \n,
/** comentario para la documentación */)
```

El análisis léxico los elimina.

Fin de entrada. Componente ficticio que indica el final de lectura.

2. OBJETIVO GENERAL

Desarrollar un programa en Lex que, dado un archivo de texto con código escrito en el lenguaje de programación C++, sea capaz de identificar y clasificar diversos elementos del lenguaje de alto nivel (como palabras reservadas, comentarios, operadores lógicos y aritméticos, e identificadores) mediante la definición de reglas para el análisis léxico.

3. OBJETIVOS ESPECÍFICOS

Diseñar las expresiones regulares necesarias en Lex para identificar las palabras reservadas de C++, los operadores aritméticos y lógicos, y los comentarios.

Implementar un analizador léxico en Lex que procese el archivo de texto, identificando y clasificando correctamente los tokens definidos.

Validar el funcionamiento del programa ejecutándolo sobre diferentes fragmentos de código en C++ para asegurar que todos los elementos del lenguaje sean reconocidos adecuadamente.

4. EQUIPO Y SOFTWARE A UTILIZAR

Equipo de cómputo con software Lex.

5. PROCEDIMIENTO PARA EL DESARROLO DE LA PRÀCTICA

Se genero el programa en LEX para que reconosca los tokens

6. Código del analizador léxico

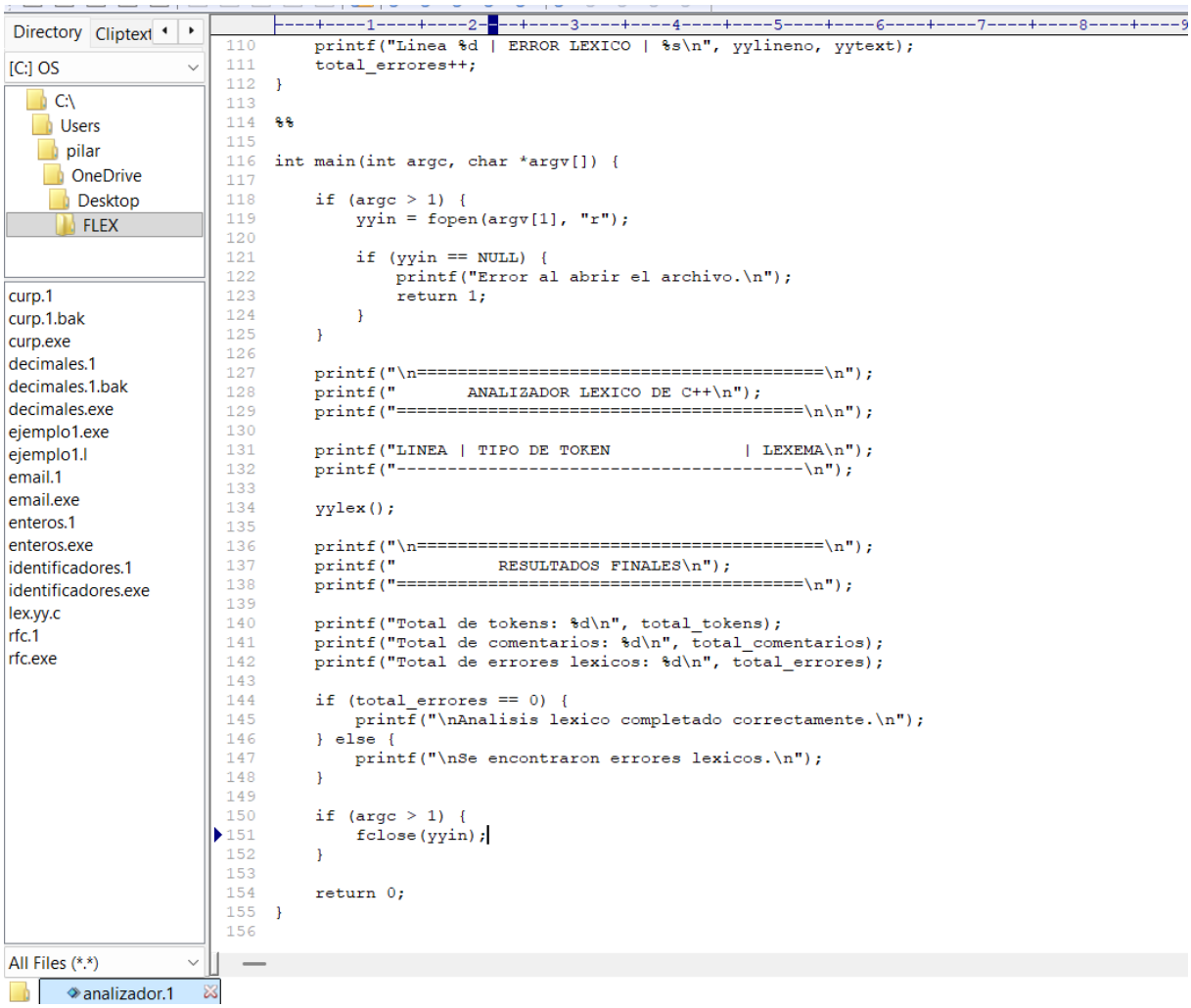
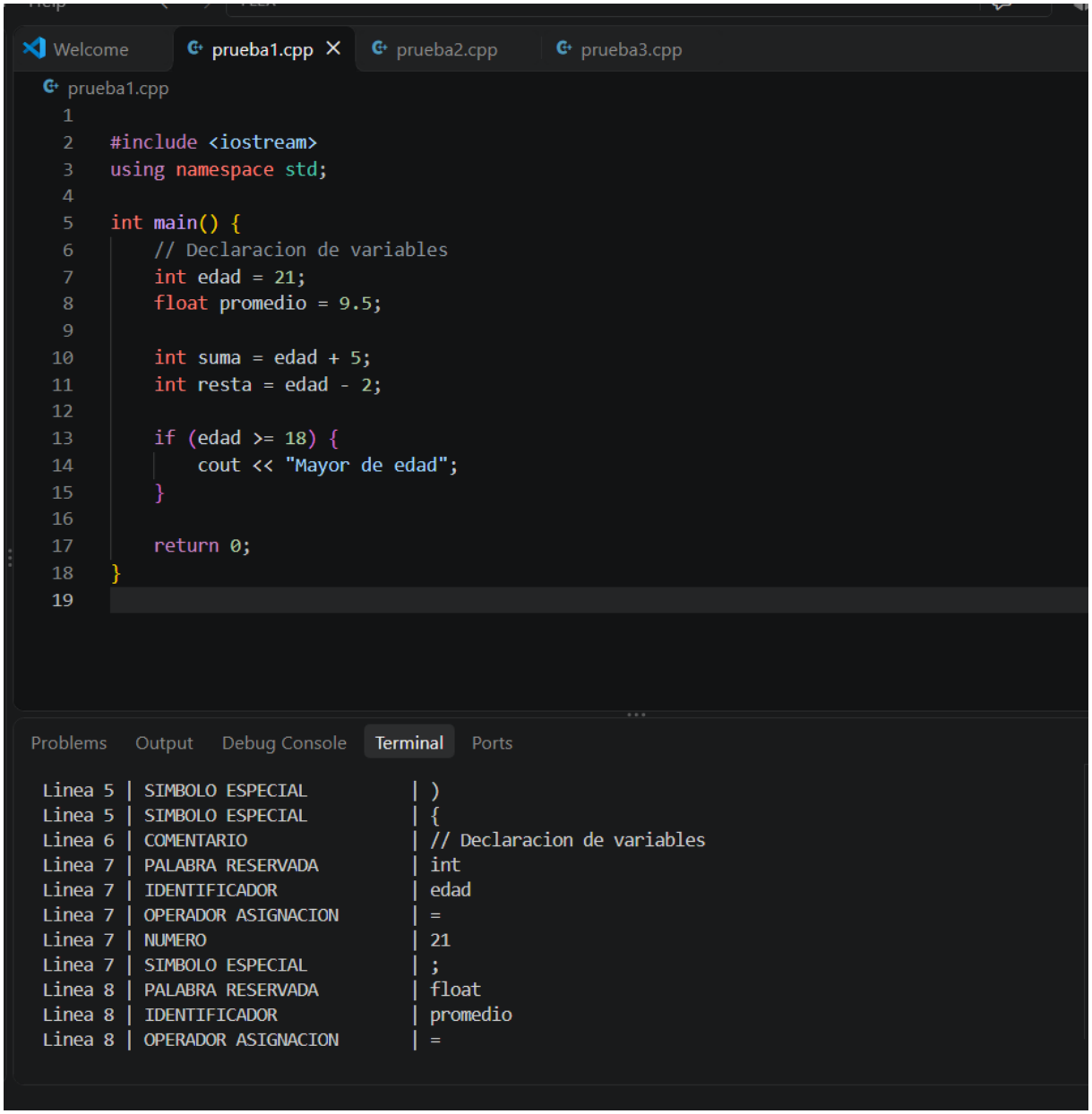
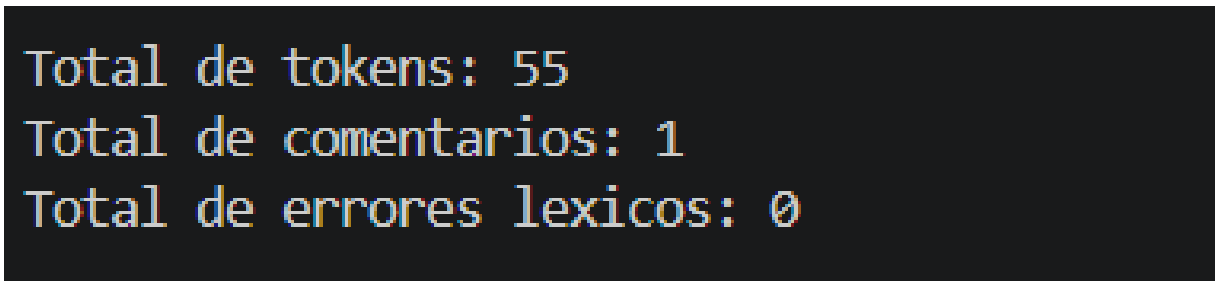


Figura 1: Código fuente del analizador léxico

En la siguiente imagen se realizo una prueba para ver si el analizador si identificaba y clasificaba los tokens

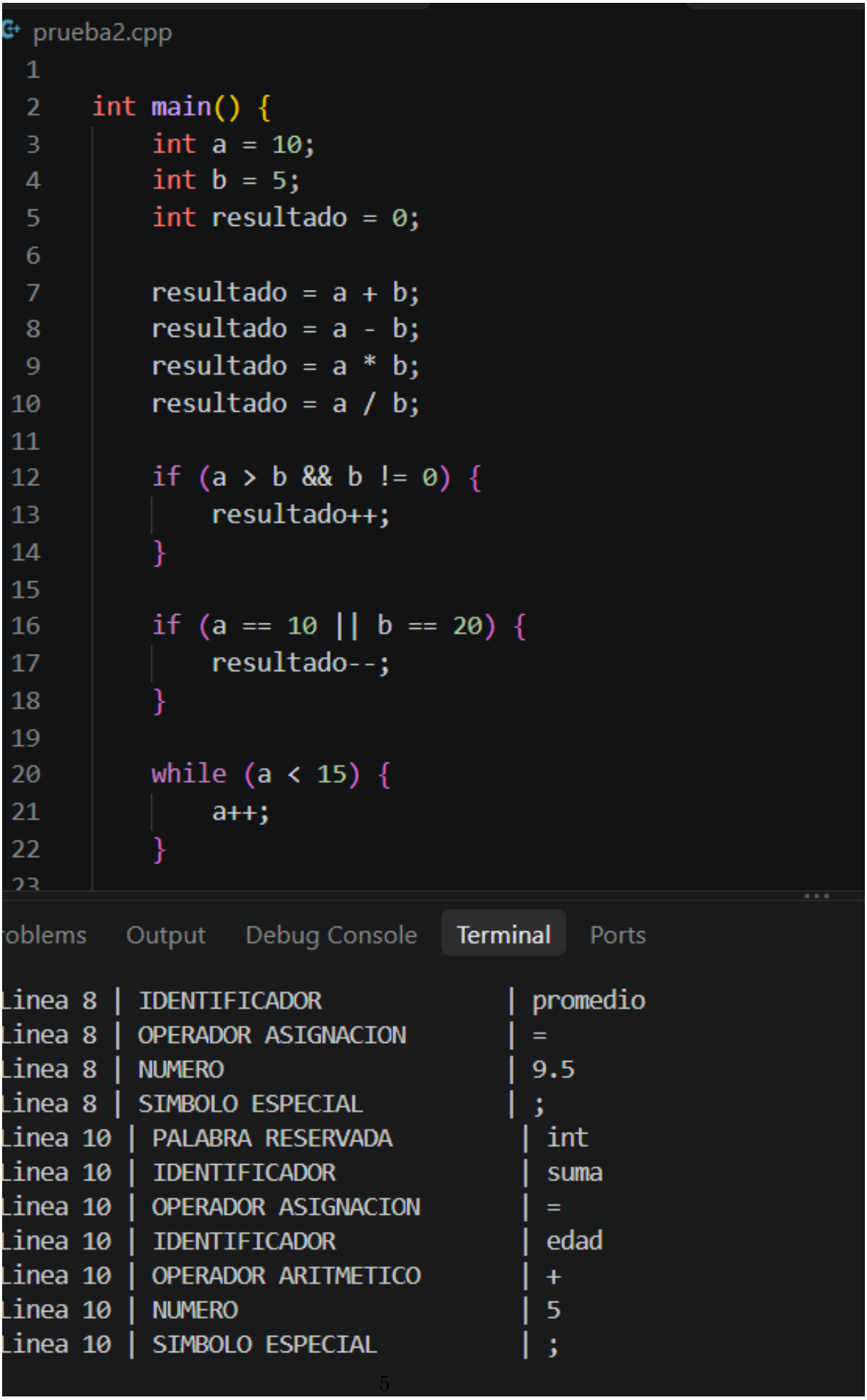


prueba 1

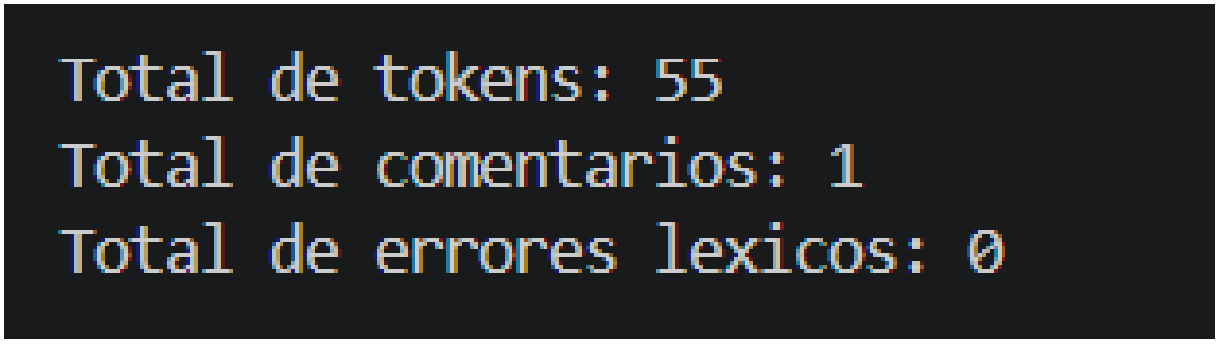


Compilacion 1

La segunda prueba es para identificar palabras revesrvadas como int, float, if, etc. identificar numero y operadores, clasificar lo sismbolos especiales.


prueba2.cpp
1
2 int main() {
3 int a = 10;
4 int b = 5;
5 int resultado = 0;
6
7 resultado = a + b;
8 resultado = a - b;
9 resultado = a * b;
10 resultado = a / b;
11
12 if (a > b && b != 0) {
13 resultado++;
14 }
15
16 if (a == 10 || b == 20) {
17 resultado--;
18 }
19
20 while (a < 15) {
21 a++;
22 }
23
oblems Output Debug Console Terminal Ports
Linea 8 | IDENTIFICADOR | promedio
Linea 8 | OPERADOR ASIGNACION | =
Linea 8 | NUMERO | 9.5
Linea 8 | SIMBOLO ESPECIAL | ;
Linea 10 | PALABRA RESERVADA | int
Linea 10 | IDENTIFICADOR | suma
Linea 10 | OPERADOR ASIGNACION | =
Linea 10 | IDENTIFICADOR | edad
Linea 10 | OPERADOR ARITMETICO | +
Linea 10 | NUMERO | 5
Linea 10 | SIMBOLO ESPECIAL | ;

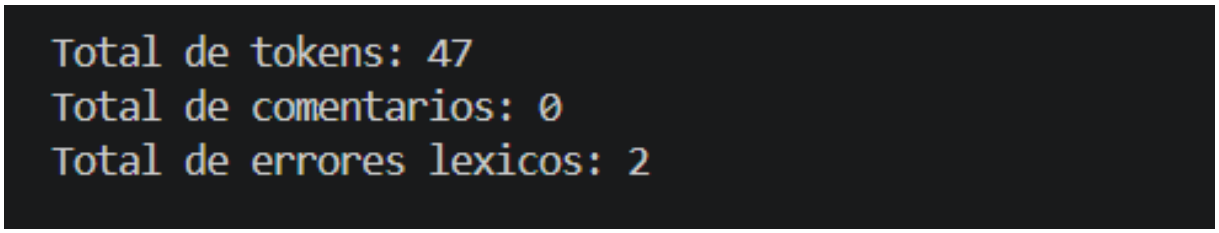
prueba 2



compilacon

beginitemize[label=]

La tercera prueba sirve para comprobar si el analizador lexico puede detectar errores cuando encuentra caracteres no correspondidos



compilacon

margin=2.5cm

Cuestionario

1. **¿Qué patrones se deben definir para identificar correctamente los tokens de números, operadores aritméticos y lógicos?**
   Se usaron patrones para reconocer números y operadores aritméticos y lógicos.
2. **¿Cuál es el proceso que seguiste para analizar las cadenas de símbolos y reconocer los tokens?**
   Se leyó el código y se comparó con las reglas para identificar los tokens.
3. **¿Qué elementos de la herramienta Lex te permitieron implementar las reglas para el análisis léxico?**
   Se utilizaron expresiones regulares y reglas de Lex para reconocer y clasificar los tokens.
4. **¿Qué condiciones o recomendaciones se deben seguir al plantear las reglas para identificar los tokens?**
   Las reglas deben ser claras y específicas para reconocer correctamente cada token.
5. **¿Cómo se logró identificar errores léxicos y qué estrategias utiliza para recuperarse de ellos?**
   Se detectaron caracteres no válidos, como @ y $, y se contaron como errores léxicos.